

BANA 4373: Applied Data Tools for Economics & Business

Lecture 7: Advanced Visualization & Dashboards

Dr. Fidel González

Sam Houston State University

Spring 2026

Agenda (and where we are in the weekly schedule)

This lecture corresponds to **Week 7: Advanced Visualization & Dashboards**.

Today we will:

- Connect static visualization (Lecture 6) to **interactive** visualization
- Learn how to build interactive plots with **Plotly**
- Introduce dashboard concepts with **Streamlit** and **Dash**
- Discuss **design for decision-makers**: labels, annotation, scale, context
- Apply skills to **inflation dashboards** (CPI components & interest rates)

Big picture: Interactive visualization enables exploration and communication with stakeholders who need to make decisions.

How Lecture 7 builds on Lecture 6

Previously, we focused on:

- Principles of effective static visualization
- Avoiding misrepresentation (axis truncation, cherry-picking)
- Creating distributions, trends, and comparisons with matplotlib/seaborn
- Applying skills to CPS labor force participation data

Today's shift:

- We move from **static** plots to **interactive** visualizations
- Users can explore data themselves (zoom, filter, hover)
- Dashboards integrate multiple views for decision-making

Key idea: Interactive visualization is not about “flashy” — it is about enabling **data-driven decisions**.

Why interactive visualization?

Static plots are powerful, but have limitations:

Static plots:

- Fixed view — one perspective per figure
- Can't explore subsets or drill down
- Limited annotation space

Interactive plots add:

- **Hover tooltips:** Show exact values without cluttering the plot
- **Zoom & pan:** Focus on specific periods or regions
- **Filtering:** Toggle categories on/off
- **Linked views:** Select in one plot, highlight in another

When to use interactive:

- Exploratory analysis (for yourself)
- Stakeholder presentations where questions arise
- Dashboards for ongoing monitoring

Introduction to Plotly

Plotly is a Python library for creating interactive visualizations.

Key features:

- Works in Jupyter notebooks, web apps, and HTML exports
- Rich interactivity: hover, zoom, pan, select
- Publication-quality output
- Easy integration with pandas DataFrames

Two main APIs:

- `plotly.express` (`px`) — Quick, high-level interface (like seaborn)
- `plotly.graph_objects` (`go`) — Full control, more verbose

Rule: Start with `plotly.express` for speed, switch to `graph_objects` for customization.

Plotly Express: Quick interactive plots

Basic syntax (very similar to seaborn):

```
import plotly.express as px

# Line chart
fig = px.line(df, x="date", y="inflation_rate",
              color="category",
              title="CPI_Inflation_by_Category")
fig.show()

# Scatter plot
fig = px.scatter(df, x="unemployment", y="inflation",
                 hover_data=["year", "fed_rate"])
fig.show()
```

Key parameters:

- `color`: Group by category
- `hover_data`: Additional info on hover
- `facet_col` / `facet_row`: Small multiples

Customizing Plotly for decision-makers

Interactive plots need even more attention to **clarity**:

```
fig.update_layout(  
    title=dict(text="<b>Monthly_CPI_Inflation</b>",  
               font_size=18),  
    xaxis_title="Date",  
    yaxis_title="Year-over-Year_Change_("%",  
    legend_title="Component",  
    hovermode="x_unified"    # All series at same x  
)  
  
fig.update_traces(  
    hovertemplate="<b>{%x}</b><br>{%y:.1f}%"  
)
```

Design principles:

- Clear, descriptive titles and axis labels
- Units in axis labels (not just in tooltips)
- Consistent number formatting

Design for decision-makers

Your audience is often **not** data scientists. They need:

Context:

- What am I looking at? (Clear titles)
- What's the baseline or benchmark?
- What time period does this cover?

Annotation:

- Highlight key events (recession, policy change)
- Call out important data points
- Add reference lines (targets, averages)

Scale:

- Is zero meaningful? Include it.
- Are comparisons fair? (Same axes for side-by-side)
- Does log scale help see patterns?

Adding annotations and reference lines

```
# Add horizontal reference line
fig.add_hline(y=2.0, line_dash="dash",
              annotation_text="Fed□Target□(2%)")

# Add vertical line for event
fig.add_vline(x="2020-03-01", line_dash="dot",
              annotation_text="COVID-19")

# Add custom annotation
fig.add_annotation(
    x="2022-06-01", y=9.1,
    text="Peak:□9.1%",
    showarrow=True, arrowhead=2
)
```

Rule: Annotations should **guide interpretation**, not clutter.

Introduction to dashboards

A **dashboard** combines multiple visualizations into a single interface.

Purpose:

- Monitor key metrics at a glance
- Enable exploration without coding
- Support recurring decisions with updated data

Dashboard components:

- **KPI cards:** Key numbers (current inflation, unemployment)
- **Time series:** Trends over time
- **Comparisons:** Bar charts, rankings
- **Filters:** Date range, category selection

Python tools:

- **Streamlit:** Fastest to build, Python-only
- **Dash (Plotly):** More control, enterprise-ready
- **Panel:** Integrates with many plotting libraries

Streamlit: Dashboards in minutes

Streamlit turns Python scripts into web apps.

```
# app.py
import streamlit as st
import plotly.express as px

st.title("Inflation Dashboard")

# Sidebar filter
category = st.sidebar.selectbox(
    "Select CPI Component",
    ["All Items", "Food", "Energy", "Core"]
)

# Filter and plot
filtered = df[df["category"] == category]
fig = px.line(filtered, x="date", y="inflation")
st.plotly_chart(fig)
```

Run with: `streamlit run app.py`

Streamlit building blocks

Display elements:

- `st.title()`, `st.header()`, `st.write()`
- `st.plotly_chart()`, `st.pyplot()`
- `st.dataframe()`, `st.table()`
- `st.metric()` — KPI cards with delta

Input widgets:

- `st.selectbox()`, `st.multiselect()`
- `st.slider()`, `st.date_input()`
- `st.checkbox()`, `st.radio()`

Layout:

- `st.sidebar` — Filters on the side
- `st.columns()` — Side-by-side content
- `st.tabs()` — Tabbed sections

Dashboard design principles

1. Start with the question, not the data

- What decision does this dashboard support?
- What do users need to see **first**?

2. Progressive disclosure

- Key metrics at the top (KPIs)
- Trends and context in the middle
- Details and drill-down at the bottom

3. Consistency

- Same colors for same categories across all charts
- Consistent date formats and axis scales
- Clear, uniform labeling

4. Don't overwhelm

- 3–5 key visualizations per view
- Use tabs or pages for additional detail

Case study: Inflation dashboard

Consumer Price Index (CPI):

- Monthly measure of price changes for consumer goods/services
- Published by Bureau of Labor Statistics (BLS)
- Includes breakdowns by component (food, energy, shelter, etc.)

Today's question:

How can we build a dashboard to monitor inflation and its components?

We will create:

- KPI cards showing current inflation rates
- Time series of headline vs. core inflation
- Component breakdown (which categories drive inflation?)
- Comparison with Federal Reserve interest rates

Mini case (in-class notebook)

In the accompanying notebook, we will:

- ➊ Load CPI inflation data (simulated, runs without downloads)
- ➋ Create interactive time series with Plotly
- ➌ Add annotations for key economic events
- ➍ Build a component breakdown chart
- ➎ Compare inflation with Fed interest rates
- ➏ (Optional) Create a simple Streamlit dashboard

Goal: Learn to create clear, interactive visualizations for economic decision-making.

Key takeaways

- Interactive visualization enables **exploration** and **communication**
- **Plotly Express** makes interactive plots as easy as seaborn
- Design for decision-makers: context, annotation, appropriate scale
- **Dashboards** combine multiple views for ongoing monitoring
- **Streamlit** is the fastest path from Python script to web app

Next week: Applied Regression in Economics — OLS as a workhorse, interpreting coefficients, diagnostics.

Questions?